Universidad Técnica Nacional

Ingeniería Electrónica

Programación II

Proyecto 1: Firmware Controlador de Banda Transportadora

Integrantes: Diego Alvarado y Valeria Solano

Profesor: Arnoldo Ramírez Jiménez

Fecha de entrega: 25 de octubre de 2026

Introducción

El presente proyecto consiste en el desarrollo de un firmware para controlar una banda transportadora mediante una placa IdeaBoard basada en el microcontrolador ESP32. El sistema utiliza un motor de corriente directa, un controlador de motor L9110S y un microinterruptor para detectar una solicitud de paro.

Para desarrollar la solución se emplean conceptos de programación de sistemas embebidos, como las máquinas de estados finitos, la generación de señales PWM, las interrupciones de hardware y los mecanismos de sincronización de FreeRTOS. También se estudia el rebote mecánico del microinterruptor y su filtrado mediante el contador de ciclos del procesador.

La investigación permite comprender los principios técnicos necesarios para implementar un sistema de control que responda a los eventos sin utilizar retardos bloqueantes en el ciclo principal del programa.

Objetivo general

Desarrollar un firmware para controlar una banda transportadora mediante una placa ESP32, utilizando una máquina de estados finitos orientada a eventos y mecanismos de control y sincronización adecuados para sistemas embebidos.

Objetivos específicos

- Investigar los mecanismos de concurrencia de FreeRTOS y su aplicación en sistemas embebidos.
- Comprender el funcionamiento de las interrupciones y las técnicas de debouncing mediante el contador de ciclos del procesador.
- Analizar el control de un motor DC mediante PWM por hardware y el controlador L9110S.
- Estudiar la implementación de una máquina de estados finitos con tiempo de ejecución constante.

- Documentar y verificar el funcionamiento del firmware mediante pruebas de integración entre software y hardware.

1. Concurrencia en FreeRTOS: Mutex, Spinlocks y Deadlocks

Pregunta 1. Contexto de tarea vs. excepción

¿Cuál es la diferencia fundamental entre el Contexto de una Tarea de FreeRTOS y el Contexto de una Excepción (ISR)? ¿Por qué invocar una función bloqueante del sistema operativo (como vTaskDelay) dentro de una ISR desencadena un pánico fatal en el núcleo?

La principal diferencia entre una tarea de FreeRTOS y una interrupción (ISR) está en la forma en que se ejecutan y en el tiempo que pueden utilizar el procesador.

Una tarea de FreeRTOS es una actividad que el sistema operativo administra mediante un planificador. Este decide qué tarea debe ejecutarse según su prioridad y disponibilidad. Además, una tarea puede detenerse temporalmente para esperar algún evento o permitir que otras tareas utilicen el procesador.

En cambio, una ISR (Interrupt Service Routine) es una función que se ejecuta cuando ocurre un evento de hardware que necesita atención. Por ejemplo, cuando se presiona un botón o cambia el estado de una entrada digital, el microcontrolador puede interrumpir temporalmente lo que estaba haciendo para atender ese evento.

La diferencia más importante es que una tarea puede esperar o bloquearse, mientras que una ISR debe ejecutarse rápidamente y terminar lo antes posible.

Por esta razón, no es correcto utilizar funciones como vTaskDelay() dentro de una interrupción. Esta función está diseñada para suspender tareas, pero una ISR no funciona como una tarea normal del sistema operativo.

Si se intenta utilizar una función bloqueante dentro de una ISR, pueden producirse errores internos, fallos de ejecución o incluso un error fatal conocido como Guru Meditation Error, dependiendo de las comprobaciones y la configuración del ESP32.

En el proyecto de la banda transportadora, esto significa que cuando se presione el microinterruptor, la interrupción solamente debe registrar que ocurrió el evento. Después, el programa principal se encargará de tomar la decisión correspondiente, como detener el motor.

De esta manera, se evita mantener ocupado innecesariamente el procesador y se consigue un funcionamiento más estable.

Pregunta 2. Spinlocks vs. Mutexes

¿Cuál es la diferencia arquitectónica entre un Mutex de tarea de FreeRTOS (SemaphoreHandle_t) y un Spinlock SMP de hardware (portMUX_TYPE)? ¿Por qué se debe usar obligatoriamente un Spinlock en lugar de un Mutex para proteger variables compartidas entre el loop() y una interrupción?

Los Mutex y los Spinlocks son mecanismos utilizados para evitar que dos partes de un programa accedan al mismo recurso al mismo tiempo. Aunque ambos buscan proteger información compartida, funcionan de maneras diferentes.

Un Mutex permite que una tarea tenga acceso exclusivo a un recurso. Si otra tarea necesita utilizarlo mientras está ocupado, puede quedar esperando hasta que se libere. Durante esa espera, el sistema operativo puede ejecutar otras tareas, por lo que el procesador no tiene que permanecer ocupado intentando acceder al recurso.

Por otro lado, un Spinlock funciona mediante espera activa. Esto significa que, si el recurso está ocupado, el procesador intenta obtenerlo repetidamente hasta que esté disponible.

En el ESP32, los Spinlocks también permiten coordinar el acceso a recursos compartidos entre sus dos núcleos. Esto es importante porque pueden existir actividades ejecutándose simultáneamente.

La razón por la que se utiliza un Spinlock en lugar de un Mutex convencional para compartir información entre el loop() y una ISR es que una interrupción no puede quedarse bloqueada esperando un recurso como lo haría una tarea normal.

Si una interrupción intentara utilizar un Mutex ocupado, podría generar problemas graves, ya que el código que debe liberar el recurso podría no tener oportunidad de ejecutarse.

Los Spinlocks utilizados por FreeRTOS, mediante portMUX_TYPE y las funciones de sección crítica correspondientes, permiten proteger operaciones muy cortas sin recurrir a los mecanismos normales de suspensión de tareas.

En este proyecto, tanto la interrupción como el programa principal necesitan acceder a información relacionada con las pulsaciones del microinterruptor. Por eso es necesario proteger esas variables para evitar que se modifiquen o consulten simultáneamente de manera incorrecta.

Es importante que el Spinlock se mantenga solamente durante el tiempo indispensable, porque una espera prolongada también puede afectar el funcionamiento del sistema.

Pregunta 3. Inversión de prioridad

Defina el concepto de Inversión de Prioridad en Sistemas Operativos de Tiempo Real. ¿Cómo previene la "Herencia de Prioridad" (Priority Inheritance) de un Mutex que el sistema se bloquee si una tarea de baja prioridad retiene un recurso que necesita una tarea de alta prioridad?

La inversión de prioridad ocurre cuando una tarea que debería ejecutarse con mayor urgencia termina esperando a otra tarea de menor prioridad.

En un sistema operativo de tiempo real, las tareas tienen prioridades que ayudan a determinar cuáles deben ejecutarse primero. Normalmente, una tarea de alta prioridad recibe atención antes que una de baja prioridad.

Sin embargo, puede presentarse una situación en la que una tarea de baja prioridad esté utilizando un recurso que necesita una tarea de alta prioridad.

Por ejemplo, imaginemos que existen tres tareas: una de prioridad alta, una de prioridad media y otra de prioridad baja.

La tarea de baja prioridad comienza a utilizar un recurso protegido por un Mutex. Poco después, la tarea de alta prioridad necesita ese mismo recurso, pero no puede acceder porque todavía está ocupado.

El problema se complica cuando la tarea de prioridad media comienza a ejecutarse y desplaza a la tarea de baja prioridad. Como resultado, la tarea de baja prioridad no puede terminar su trabajo ni liberar el recurso, mientras que la tarea de alta prioridad continúa esperando.

Para solucionar este problema existe un mecanismo llamado Priority Inheritance, o herencia de prioridad.

Este mecanismo permite que la tarea de baja prioridad reciba temporalmente una prioridad más alta cuando está reteniendo un recurso que necesita una tarea más importante.

Así, la tarea puede terminar rápidamente la operación, liberar el recurso y permitir que la tarea de alta prioridad continúe.

Una vez liberado el Mutex, la prioridad temporal puede regresar a su valor correspondiente.

La herencia de prioridad ayuda a reducir los retrasos provocados por la inversión de prioridad. Sin embargo, no elimina todos los tipos de bloqueo que pueden presentarse en un sistema.

En el proyecto de la banda transportadora, comprender este concepto es importante porque el sistema debe responder oportunamente a los eventos, especialmente cuando se necesita detener el motor.

Pregunta 4. Condiciones de Deadlock

Enumere las cuatro condiciones de Coffman para que exista un Abrazo Mortal (Deadlock). Describa la secuencia exacta de eventos que causaría un Deadlock si el bucle principal retiene un Mutex y una ISR intenta adquirir ese mismo Mutex.

Un Deadlock, también conocido como abrazo mortal, es una situación en la que dos o más tareas quedan esperando recursos y ninguna puede continuar con su ejecución.

Para que ocurra un Deadlock clásico, deben presentarse simultáneamente cuatro condiciones, conocidas como las condiciones de Coffman.

1. Exclusión mutua: significa que un recurso solamente puede ser utilizado por una tarea a la vez. Si otra tarea lo necesita, debe esperar hasta que esté disponible.

2. Retención y espera: ocurre cuando una tarea mantiene un recurso mientras espera conseguir otro que necesita para continuar.

3. Ausencia de expropiación: significa que un recurso no puede quitarse por la fuerza a la tarea que lo está utilizando. La tarea debe liberarlo voluntariamente.

4. Espera circular: sucede cuando existe una cadena de tareas que esperan recursos retenidos por otras, formando un ciclo del cual ninguna puede salir.

En el caso del proyecto, imaginemos que el programa principal adquiere un Mutex para trabajar con una variable compartida.

Mientras el Mutex sigue ocupado, el usuario presiona el microinterruptor y se genera una interrupción.

La ISR comienza a ejecutarse e intenta adquirir ese mismo Mutex. Sin embargo, el recurso todavía está siendo utilizado por el programa principal.

Si la interrupción intenta esperar hasta que el Mutex quede libre, aparece un problema: el programa principal no puede continuar y liberar el recurso mientras la interrupción lo mantiene interrumpido en ese núcleo.

Como resultado, el sistema podría quedar bloqueado.

Técnicamente, este caso no necesariamente cumple las cuatro condiciones de Coffman de un Deadlock clásico entre tareas, pero sí representa una situación de bloqueo causada por utilizar incorrectamente un mecanismo de sincronización dentro de una ISR.

Por esta razón, las interrupciones no deben utilizar Mutexes convencionales ni realizar operaciones que puedan bloquearse.

En la banda transportadora, evitar estos problemas es fundamental, ya que un bloqueo podría impedir que el sistema responda correctamente ante una pulsación del botón de paro.

2. Arquitectura de memoria y restricciones del silicio

Pregunta 5. Caché de Flash e IRAM_ATTR

El ESP32 ejecuta código desde una memoria Flash SPI externa a través de un controlador de caché. ¿Qué le ocurre a esta caché cuando el sistema escribe datos en la memoria no volátil (NVS) o realiza una actualización OTA? ¿Por qué omitir la macro IRAM_ATTR en la ISR causa un colapso del sistema (Guru Meditation Error) durante estas operaciones?

El ESP32 utiliza memoria Flash externa para almacenar el programa y otros datos que deben conservarse incluso cuando se desconecta la alimentación.

Como acceder directamente a esta memoria puede ser más lento, el procesador utiliza una memoria caché. Su función es guardar temporalmente información que se utiliza con frecuencia para que el acceso sea más rápido.

Sin embargo, cuando el ESP32 necesita escribir o borrar información en la memoria Flash, como sucede durante algunas operaciones de NVS (Non-Volatile Storage) o durante una actualización OTA (Over-The-Air), el acceso a la caché de Flash puede deshabilitarse temporalmente.

Esto significa que algunas instrucciones almacenadas en la memoria externa no estarán disponibles durante ese intervalo.

El problema aparece cuando ocurre una interrupción que intenta ejecutar instrucciones ubicadas en Flash mientras la caché está deshabilitada.

Para evitar esta situación se utiliza IRAM_ATTR.

Esta macro permite colocar la función de interrupción en la memoria RAM interna destinada a instrucciones, conocida como IRAM.

La ventaja es que esa memoria puede seguir siendo accesible aunque la caché de Flash se encuentre temporalmente deshabilitada.

Si una interrupción configurada para ejecutarse en esas condiciones intenta acceder a código que no está disponible, puede producirse un fallo del sistema, conocido como Guru Meditation Error.

No obstante, es importante aclarar que omitir IRAM_ATTR no siempre produce un error. Algunas interrupciones pueden ser deshabilitadas temporalmente durante las operaciones de Flash. El problema depende de cómo esté configurada la interrupción y de los recursos a los que accede.

Además, no basta con colocar la función principal en IRAM; también deben ser accesibles las funciones y los datos utilizados por ella.

En el proyecto de la banda transportadora, utilizar correctamente IRAM_ATTR ayuda a que la rutina del microinterruptor pueda ejecutarse de forma segura bajo las condiciones de memoria para las que fue diseñada.

Pregunta 6. Lecturas fragmentadas (Torn Reads)

El ESP32 posee un bus de datos de 32 bits. Si una ISR actualiza un contador de 64 bits (uint64_t), ¿por qué su lectura en el bucle principal requiere dos instrucciones máquina de 32 bits separadas? Explique cómo una interrupción que se dispare exactamente entre estas dos instrucciones provoca una lectura corrupta (Torn Read).

El ESP32 clásico trabaja con una arquitectura de 32 bits. Esto significa que sus registros y muchas de sus operaciones están diseñados para manejar datos de ese tamaño.

Cuando se utiliza una variable de 64 bits, como un contador uint64_t, el procesador puede necesitar dividir su lectura en dos operaciones de 32 bits.

Primero se lee una parte del número y después la otra.

El problema aparece cuando una interrupción modifica el contador justo entre esas dos lecturas.

Por ejemplo, imaginemos que el programa principal comienza a leer un contador de 64 bits. Primero obtiene la mitad inferior del valor.

Antes de que pueda leer la mitad superior, ocurre una interrupción que incrementa el contador.

Cuando el programa principal continúa, obtiene la segunda mitad, pero esta ya pertenece al valor actualizado.

Por lo tanto, termina combinando dos partes que corresponden a momentos diferentes.

El resultado puede ser un número incorrecto que nunca estuvo realmente almacenado en la variable.

A este problema se le conoce como Torn Read, o lectura fragmentada.

Para evitarlo, es necesario utilizar mecanismos de sincronización que impidan que la variable cambie mientras se realiza su lectura.

En el proyecto se utilizan Spinlocks para proteger el acceso a las variables compartidas.

Esto es especialmente importante cuando se lleva un registro de las pulsaciones del microinterruptor, porque una lectura incorrecta podría ocasionar errores en el conteo de eventos.

Pregunta 7. La palabra clave volatile

Explique la optimización de Asignación de Registros (Register Allocation) del compilador bajo el nivel -O2. Si una bandera no se declara como volatile, ¿por qué el loop() podría evaluar un registro obsoleto de la CPU (como el registro a2) ignorando permanentemente los cambios físicos en la RAM realizados por la ISR?

Cuando se compila un programa, el compilador puede aplicar optimizaciones para mejorar su velocidad y reducir el número de instrucciones que debe ejecutar el procesador.

Una de estas optimizaciones es la asignación de registros, conocida como Register Allocation.

Esta técnica consiste en mantener ciertos valores dentro de los registros internos del procesador, en lugar de acceder constantemente a la memoria RAM.

Con niveles de optimización como -O2, el compilador puede considerar que no es necesario volver a leer una variable si aparentemente no ha cambiado.

Esto normalmente mejora el rendimiento, pero puede causar problemas cuando una variable es modificada por una interrupción.

Por ejemplo, imaginemos que el programa principal revisa constantemente una bandera para saber si se presionó el microinterruptor.

Si esa bandera no se declara correctamente, el compilador podría conservar una copia de su valor en un registro del procesador, como a2, y reutilizarla sin consultar nuevamente la memoria RAM.

Mientras tanto, la interrupción podría modificar el valor real de la bandera.

El problema es que el programa principal podría seguir utilizando el valor anterior, sin reconocer que ocurrió una pulsación.

Para evitar este comportamiento se utiliza la palabra clave volatile.

Esta indica al compilador que el valor de una variable puede cambiar de manera externa al flujo normal del programa y que sus accesos no deben eliminarse como si fueran innecesarios.

Sin embargo, volatile no garantiza que una operación sea atómica ni evita por sí sola las condiciones de carrera.

Por eso, cuando una variable es compartida entre el programa principal y una interrupción, también pueden ser necesarios mecanismos de sincronización.

En la banda transportadora, volatile ayuda a que el programa principal observe los cambios en las banderas de eventos generados por la interrupción, mientras que los Spinlocks protegen los accesos compartidos.

3. Física del rebote mecánico y depuración (Debouncing)

Pregunta 8. Física del contacto

¿Qué ocurre físicamente a nivel microscópico cuando los contactos metálicos de un microinterruptor se cierran? ¿Por qué una sola pulsación humana es interpretada por un microcontrolador de 160 MHz como ráfagas de 20 a 50 cambios de nivel lógico?

Respuesta:

Cuando una persona presiona un microinterruptor, normalmente piensa que el contacto eléctrico se cierra de manera inmediata. Sin embargo, físicamente el proceso es un poco más complejo.

Los contactos metálicos del interruptor no son completamente lisos. A nivel microscópico presentan pequeñas irregularidades que influyen en la forma en que se tocan.

Cuando se presiona el interruptor, las superficies metálicas chocan entre sí. Debido a la elasticidad de los materiales y a la energía del impacto, los contactos pueden separarse y volver a unirse varias veces antes de quedar completamente estables.

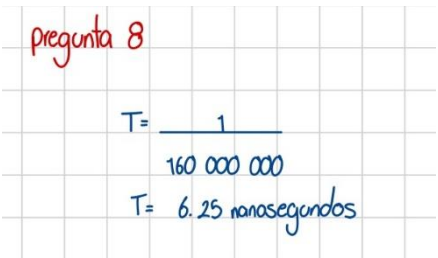
Este fenómeno se conoce como rebote mecánico o Contact Bounce.

Aunque una persona solamente presione el botón una vez, eléctricamente pueden producirse varios cambios rápidos entre los niveles alto y bajo.

El ESP32 puede detectar estas variaciones porque trabaja a velocidades mucho mayores que las de los movimientos humanos.

Por ejemplo, si el procesador funciona a 160 MHz, realiza 160 millones de ciclos de reloj por segundo.

El tiempo correspondiente a un ciclo es:



Esto demuestra que el procesador puede trabajar con intervalos extremadamente pequeños.

Por esa razón, los rebotes que ocurren durante unos pocos milisegundos pueden generar numerosas transiciones detectables por el microcontrolador.

La cantidad de 20 a 50 cambios mencionada en el enunciado es un ejemplo de lo que puede ocurrir, aunque el número real depende de las características del interruptor.

Si no se aplica un método de filtrado, el sistema podría interpretar una sola pulsación como muchas pulsaciones diferentes.

En la banda transportadora, esto podría ocasionar que se registren eventos adicionales que el usuario realmente no realizó.

Para evitarlo, se utiliza una técnica llamada Debouncing, cuya función es filtrar los rebotes y reconocer una pulsación válida.

Pregunta 9. Depuración por ciclos (CCOUNT)

¿Por qué el uso de millis() o delay() dentro de una ISR constituye una mala práctica de ingeniería? ¿De qué manera el registro de hardware XTHAL_GET_CCOUNT() provee una marca de tiempo en un solo ciclo de reloj (non-blocking)?

Cuando se trabaja con interrupciones, es importante que las operaciones se realicen lo más rápido posible.

Una interrupción debe atender el evento que la activó y regresar al funcionamiento normal del sistema, sin generar esperas innecesarias.

Por esta razón, utilizar funciones como delay() dentro de una ISR es una mala práctica, ya que introduce una espera que mantiene ocupada la ejecución y puede afectar la atención de otros eventos.

En el caso de millis(), la situación es diferente porque esta función no introduce necesariamente un retardo. Sin embargo, trabaja con una resolución de milisegundos y depende de los mecanismos de temporización del entorno Arduino.

Para este proyecto se solicita una alternativa basada directamente en el hardware: el registro CCOUNT.

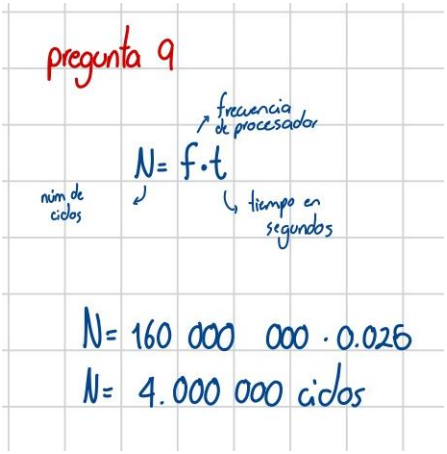
Este registro lleva un conteo de los ciclos del procesador. Mediante XTHAL_GET_CCOUNT() es posible consultar su valor actual de forma rápida, sin tener que detener la ejecución para esperar que transcurra un tiempo.

La idea es guardar el valor del contador cuando ocurre una pulsación y compararlo con el valor obtenido durante la siguiente interrupción.

Si la diferencia entre ambos valores es suficientemente grande, se considera que ocurrió una nueva pulsación válida.

El proyecto establece un intervalo de filtrado de 25 milisegundos.

Para calcular cuántos ciclos corresponden a ese tiempo, se utiliza:



Esto significa que deben transcurrir al menos cuatro millones de ciclos para aceptar una nueva pulsación.

La ventaja de este método es que no necesita detener el programa durante 25 milisegundos. Simplemente compara dos valores del contador.

Por eso se considera un método no bloqueante.

En el proyecto, esta técnica ayuda a evitar que los rebotes del microinterruptor generen múltiples eventos no deseados.

El cálculo supone que la frecuencia de la CPU se mantiene en 160 MHz y que las marcas temporales utilizadas corresponden al mismo contador de núcleo.

Pregunta 10. Aritmética modular

El registro de 32 bits CCOUNT se desborda y vuelve a cero (rollover) cada 26,84 segundos. Demuestre matemáticamente por qué la resta de enteros sin signo de 32 bits (currentCycles - isrLastCycle) calcula el tiempo transcurrido de forma perfecta sin importar si ocurre un desbordamiento.

El registro CCOUNT del ESP32 clásico tiene un tamaño de 32 bits. Esto significa que puede representar una cantidad limitada de valores.

El número total de valores posibles es:



Por lo tanto, el contador puede avanzar desde cero hasta:



Cuando llega al valor máximo, vuelve a comenzar desde cero.

Este comportamiento se conoce como rollover o desbordamiento.

Si el procesador funciona a 160 MHz, el tiempo que tarda el contador en completar una vuelta es:

$$T = \frac{2^{32}}{160\,000\,000}$$

T= 26.84 s

A primera vista, podría parecer que este desbordamiento causaría errores al calcular el tiempo transcurrido entre dos eventos.

Sin embargo, los enteros sin signo de 32 bits utilizan aritmética modular, lo que permite manejar correctamente este comportamiento.

La diferencia entre dos valores se expresa como:

$$\Delta C = (C_{actual} - C_{anterior}) \bmod 2^{32}$$

Para comprenderlo, supongamos que el contador anterior tenía el siguiente valor:

$$C_{anterior} = 4\,294\,967\,000$$

Después de cierto tiempo, el contador se desborda y su nuevo valor es:

$$C_{actual} = 600$$

Si se realiza una resta convencional, se obtiene un número negativo. Pero al utilizar aritmética modular de 32 bits:

$$\Delta C = (500 - 4294967000) \bmod 4294967000$$

El resultado es:

$$\Delta C = 796 \text{ ciclos}$$

Esto significa que transcurrieron 796 ciclos entre ambas lecturas, aunque el contador haya vuelto a cero.

La razón es que la aritmética modular conserva correctamente la diferencia dentro del rango de representación de 32 bits.

No obstante, existe una condición importante: el intervalo real debe ser menor que una vuelta completa del contador. Si transcurren 26.84 segundos o más, la diferencia modular no permite saber cuántas vueltas completas ocurrieron.

En el caso del proyecto, el intervalo que se desea medir es de solamente 25 milisegundos, por lo que este método resulta adecuado bajo las condiciones normales de funcionamiento.

Gracias a esto, el sistema puede filtrar las pulsaciones sin necesidad de reiniciar manualmente el contador cada vez que ocurre un desbordamiento.

4. Periféricos de la IdeaBoard y determinismo algorítmico

Pregunta 11. Hardware PWM

¿Por qué es necesario configurar el hardware timer LEDC_TIMER_0 con una resolución de 10 bits y una frecuencia específica antes de asignar los canales para controlar el puente H L9110S en los pines IO12 e IO14?

Para controlar la velocidad de un motor de corriente directa, es común utilizar una técnica llamada PWM, que significa Pulse Width Modulation o modulación por ancho de pulso.

Esta técnica consiste en encender y apagar una señal eléctrica de manera muy rápida, modificando el tiempo durante el cual permanece activa.

La proporción entre el tiempo encendido y el período total se conoce como ciclo de trabajo o Duty Cycle.

Cuando el ciclo de trabajo aumenta, también aumenta la tensión media aplicada al motor bajo condiciones ideales. Esto permite influir en su velocidad de giro.

El ESP32 cuenta con un periférico llamado LEDC, diseñado para generar señales PWM mediante hardware.

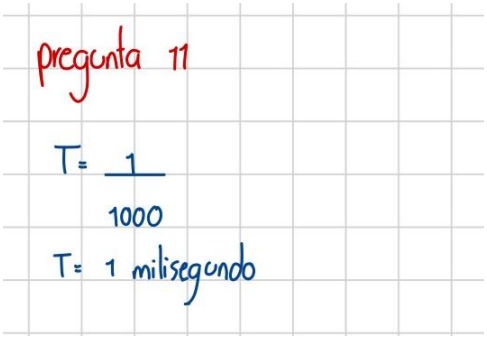
Una de sus ventajas es que puede mantener la generación de pulsos sin que el programa principal tenga que cambiar manualmente el estado de los pines todo el tiempo.

Antes de utilizar los canales PWM, es necesario configurar el temporizador que establece las características de la señal.

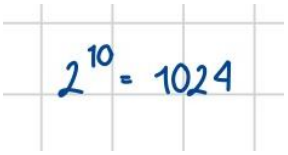
En este proyecto se solicita una frecuencia de 1000 Hz y una resolución de 10 bits.

La frecuencia de 1000 Hz significa que la señal completa mil ciclos por segundo.

El período de cada ciclo es:



Por otra parte, la resolución de 10 bits permite representar:



niveles de cuantización para el control del ciclo de trabajo.

Esto proporciona un ajuste más detallado que una resolución de menor cantidad de bits.

La configuración del temporizador debe realizarse antes de asociar los canales porque estos necesitan conocer la frecuencia y resolución con las que van a funcionar.

Posteriormente, las señales se utilizan para controlar el puente H L9110S, conectado a los pines IO12 e IO14 de la IdeaBoard.

Este controlador permite manejar el comportamiento del motor, incluyendo su movimiento y las condiciones de frenado o giro por inercia, de acuerdo con las combinaciones de sus entradas.

En la banda transportadora, esta configuración es importante porque permite controlar el motor mediante señales generadas directamente por el hardware, manteniendo una operación más estable y reduciendo el trabajo del procesador.

Pregunta 12. Resistencias Pull-up

Si el interruptor de paro de emergencia se trasladara del pin IO33 al pin IO34, ¿por qué el circuito dejaría de funcionar por completo a menos que se soldara una resistencia externa?

Los pines digitales de un microcontrolador necesitan recibir niveles eléctricos definidos para poder interpretar correctamente si una entrada está en estado alto o bajo.

Cuando un interruptor se conecta entre un pin y tierra, el pin recibe un nivel bajo cuando el interruptor está cerrado.

Sin embargo, cuando el interruptor está abierto, la entrada puede quedar sin una conexión que establezca claramente su nivel lógico.

A esto se le conoce como entrada flotante.

Una entrada flotante puede captar interferencias eléctricas y producir lecturas incorrectas, ya que el microcontrolador no puede determinar de manera confiable si el nivel es alto o bajo.

Para evitar este problema se utilizan resistencias pull-up.

Una resistencia pull-up conecta la entrada a una tensión positiva mediante una resistencia, de forma que el pin permanezca normalmente en nivel alto.

Cuando se presiona el interruptor y se conecta el pin a tierra, el nivel cambia a bajo.

En el ESP32 clásico, el GPIO33 permite utilizar una resistencia pull-up interna, por lo que no es necesario agregar una resistencia externa para conseguir esta función.

Sin embargo, el GPIO34 tiene una característica diferente: es un pin de entrada que no dispone de resistencia pull-up interna.

Esto significa que, si el interruptor se trasladara al GPIO34 sin modificar el circuito, el pin podría quedar flotante cuando el interruptor estuviera abierto.

Como consecuencia, el sistema podría detectar cambios falsos o no reconocer correctamente el estado del interruptor.

Para solucionar el problema, sería necesario conectar una resistencia pull-up externa entre el GPIO34 y una alimentación lógica compatible, normalmente de 3.3 V.

Así se garantiza que la entrada tenga un nivel alto cuando el interruptor esté abierto y un nivel bajo cuando se presione.

Por lo tanto, aunque el GPIO34 puede utilizarse como entrada digital, necesita una resistencia externa para implementar correctamente este tipo de conexión.

En el proyecto se utiliza GPIO33 precisamente porque permite aprovechar la resistencia pull-up interna y simplificar el circuito del microinterruptor.

Pregunta 13. Matrices de transición y complejidad O(1)

El proyecto mapea Estados (Filas) y Eventos (Columnas) utilizando un arreglo bidimensional de punteros a funciones (constexpr Transition FSM[][]). ¿Por qué el tiempo de evaluación de esta matriz se considera de complejidad constante O(1)? ¿Por qué esto es superior a múltiples sentencias anidadas de switch-case en términos de predicción de bifurcaciones del procesador (Branch Prediction)?

Una Máquina de Estados Finitos, conocida como FSM (Finite State Machine), es una forma de organizar el comportamiento de un sistema mediante estados y eventos.

Cada estado representa una condición en la que puede encontrarse el sistema, mientras que los eventos representan situaciones que pueden provocar un cambio.

Por ejemplo, en una banda transportadora se pueden definir estados como detenido, funcionando y fallo.

Cuando ocurre un evento, como la pulsación de un interruptor, el sistema analiza el estado actual y determina qué acción corresponde realizar.

Una forma de organizar estas decisiones es mediante una matriz de transición.

En esta matriz, las filas representan los estados y las columnas representan los eventos. Cada posición contiene una referencia a la función encargada de atender esa combinación.

La principal ventaja es que el sistema puede encontrar directamente la acción correspondiente utilizando los índices del estado y del evento.

Esto se considera una operación de complejidad constante, representada mediante la notación O(1).

La complejidad constante significa que el número de operaciones necesarias para acceder a una posición de la matriz no aumenta proporcionalmente con la cantidad de estados o eventos.

Para comprenderlo, imaginemos que la matriz contiene tres estados y tres eventos.

El sistema no necesita revisar una por una todas las combinaciones posibles. Simplemente utiliza los índices correspondientes y accede directamente a la posición deseada.

La dirección de una posición puede calcularse mediante:

Dirección=Base+i* N+j)*S

Donde:

- I representa el índice del estado.
- j representa el índice del evento.
- Nes el número de columnas.
- S es el tamaño de cada elemento.

Gracias a este mecanismo, la selección de la función correspondiente puede realizarse mediante una cantidad fija de operaciones.

Por otro lado, cuando se utilizan muchas sentencias if-else o switch-case anidadas, la lógica puede volverse más difícil de comprender y mantener.

El programa debe evaluar las condiciones necesarias para determinar qué acción ejecutar, aunque los compiladores modernos pueden optimizar muchas de estas estructuras.

La predicción de bifurcaciones (Branch Prediction) es un mecanismo que algunos procesadores utilizan para anticipar el camino que seguirá un programa cuando encuentra instrucciones condicionales.

Sin embargo, no sería correcto afirmar que una matriz de funciones siempre ofrece una mejor predicción de bifurcaciones o que siempre es más rápida que un switch-case.

Un compilador puede transformar un switch-case en una tabla de saltos, consiguiendo también un despacho de complejidad constante.

Además, una llamada mediante un puntero a función puede tener costos distintos a los de una llamada directa.

Por lo tanto, la principal ventaja de utilizar una matriz de transición en este proyecto no es garantizar que siempre sea la opción más rápida, sino conseguir una estructura más ordenada, fácil de mantener y con un tiempo de selección predecible.

Esto es especialmente importante en sistemas embebidos, donde se busca que el comportamiento del programa sea claro y que las respuestas a los eventos se realicen sin retardos innecesarios.

En la banda transportadora, la matriz FSM permite organizar las decisiones relacionadas con el motor según el estado actual y el evento recibido, evitando que la lógica principal se convierta en una gran cantidad de condiciones anidadas.